

Playwright XPath Locators — Complete Notes

Updated notes covering XPath locator techniques, dynamic XPath patterns, element collections, text matching, and important Playwright corrections.

1. Relative XPath

Relative XPath begins with `//` and searches for elements based on their attributes, text, or relationships.

```
const logo: Locator = page.locator("//img[@alt='Tricentis Demo Web Shop']");
await expect(logo).toBeVisible();
```

2. Absolute XPath

Absolute XPath starts from the root of the HTML document using a single `/`. It follows the complete DOM hierarchy and can be fragile when the page structure changes.

```
const abspath: Locator = page.locator(
  "/html/body/div[4]/div[1]/div[1]/div[1]/a/img"
);
await expect(abspath).toBeVisible();
```

3. contains()

`contains()` is useful for dynamic or partially known attribute values.

```
//*[contains(@class, 'btn')]

const products: Locator =
  page.locator("//h2/a[contains(@href, 'computer')]");
```

4. count(), first(), nth(), allTextContents()

```
const counter: number = await products.count();
expect(counter).toBeGreaterThan(0);

console.log("First:", await products.first().textContent());
console.log("Fourth:", await products.nth(3).textContent());

const productTitles: string[] = await products.allTextContents();
for (let pt of productTitles) {
  console.log(pt);
}
```

Remember: Playwright uses zero-based indexing. `nth(0)` is the first element and `nth(3)` is the fourth.

5. starts-with()

`starts-with()` checks whether an attribute begins with a specific value. It is useful when the beginning of a dynamic URL, ID, class, or other attribute is stable.

```
const buildingProduct: Locator = page.locator(
  "//h2/a[starts-with(@href, '/build')]"
);
const count = await buildingProduct.count();
expect(count).toBeGreaterThan(0);
```

6. text()

```
const button1: Locator = page.locator("//a[text()='Register']");
await expect(button1).toBeVisible();
```

`text()` can be used to match the text node of an element. This example targets an anchor whose text is exactly **Register**.

7. normalize-space()

```
const button2: Locator =  
  page.locator("//a[normalize-space()='Register']");  
await expect(button2).toBeVisible();
```

`normalize-space()` removes leading/trailing whitespace and collapses consecutive whitespace. It is helpful when HTML contains unexpected spaces or line breaks.

8. Dynamic XPath — START / STOP Example

The uploaded example demonstrates how the same element type can be located dynamically using different XPath techniques. The page contains two buttons: **START** and **STOP**.

Dynamic XPath



```
<button name="start" class="start" style="">START</button>
```

```
<button name="stop" class="stop" style="">STOP</button>
```

XPath	Technique
<code>//button[@name='start']</code>	Exception
<code>//button[@name='stop']</code>	Exception
<code>//button[@name='start' or @name='stop']</code>	or
<code>//button[contains(@name,'st')]</code>	contains()
<code>//button[starts-with(@name,'st')]</code>	starts-with()
<code>//button[text()='STOP' or text()='START'];</code>	text() - innertext

9. Dynamic XPath Techniques from the Example

The example shows several ways to locate either the START or STOP button. The key idea is that instead of writing two completely separate locators, you can sometimes create one locator that handles multiple possible values.

XPath	Technique / Purpose
<code>//button[@name='start']</code>	Exact attribute match for START
<code>//button[@name='stop']</code>	Exact attribute match for STOP
<code>//button[@name='start' or @name='stop']</code>	OR operator — match either START or STOP
<code>//button[contains(@name,'st')]</code>	contains() — match a partial attribute value
<code>//button[starts-with(@name,'st')]</code>	starts-with() — match an attribute beginning with a value
<code>//button[text()='STOP' or text()='START']</code>	text() + OR — match either button by visible text

10. OR Operator in XPath

The XPath or operator lets you match more than one condition.

```
//button[@name='start' or @name='stop']
```

This matches a button when its name attribute is either start or stop.

11. contains() vs starts-with()

contains() checks whether the value contains a particular piece of text anywhere. **starts-with()** checks specifically whether the value begins with that text.

```
// Contains
//button[contains(@name, 'st')]

// Starts with
//button[starts-with(@name, 'st')]
```

12. text() vs normalize-space()

text() is useful when the exact text node is known. **normalize-space()** is more tolerant of extra spaces and line breaks.

```
// Exact text
//a[text()='Register']

// Text after whitespace normalization
//a[normalize-space()='Register']
```

13. Important Corrections in the Code

When checking the absolute XPath locator, the assertion should use `abspath`, not `logo`:

```
const abspath: Locator = page.locator(
  "/html/body/div[4]/div[1]/div[1]/div[1]/a/img"
);
await expect(abspath).toBeVisible();
```

Also use `await expect(...).toBeVisible()` for the asynchronous Playwright assertion.

14. Complete Cleaned-Up Code

```
import { test, expect, Locator } from "@playwright/test";

test("Xpath Locators", async ({ page }) => {

  await page.goto("https://demowebshop.tricentis.com/");
  await page.waitForTimeout(3000);

  // 1. Relative XPath
  const logo: Locator =
    page.locator("//img[@alt='Tricentis Demo Web Shop']");
  await expect(logo).toBeVisible();

  // 2. Absolute XPath
  const abspath: Locator =
    page.locator("/html/body/div[4]/div[1]/div[1]/div[1]/a/img");
  await expect(abspath).toBeVisible();

  // 3. contains()
  const products: Locator =
    page.locator("//h2/a[contains(@href,'computer')]");

  const counter: number = await products.count();
  console.log("No of computer related products", counter);
  expect(counter).toBeGreaterThan(0);

  console.log(
    "First computer related product",
```

```

    await products.first().textContent()
  );

  console.log(
    "Fourth computer related product",
    await products.nth(3).textContent()
  );

  const productTitles: string[] =
    await products.allTextContents();

  for (let pt of productTitles) {
    console.log(pt);
  }

  // 4. starts-with()
  const buildingProduct: Locator =
    page.locator("//h2/a[starts-with(@href, '/build')]");

  const buildCount = await buildingProduct.count();
  expect(buildCount).toBeGreaterThan(0);

  // 5. text()
  const button1: Locator =
    page.locator("//a[text()='Register']");
  await expect(button1).toBeVisible();

  // 6. normalize-space()
  const button2: Locator =
    page.locator("//a[normalize-space()='Register']");
  await expect(button2).toBeVisible();

  // Dynamic XPath examples:
  // //button[@name='start' or @name='stop']
  // //button[contains(@name, 'st')]
  // //button[starts-with(@name, 'st')]
  // //button[text()='STOP' or text()='START']
});

```

15. Quick Revision Cheat Sheet

Technique	Example	Purpose
Relative XPath	//img[@alt='...']	Locate using a relative DOM path
Absolute XPath	/html/body/...	Full DOM path; fragile
contains()	contains(@href,'computer')	Partial match
starts-with()	starts-with(@href,'/build')	Prefix match
OR	@name='start' or @name='stop'	Match either condition
text()	text()='Register'	Match exact text
normalize-space()	normalize-space()='Register'	Handle extra whitespace
count()	await locator.count()	Count matches
first()	locator.first()	First match
nth()	locator.nth(3)	Specific zero-based index
allTextContents()	await locator.allTextContents()	Get text of all matches

Best practice: Prefer Playwright's user-facing locators such as `getByRole()`, `getByLabel()`, `getByText()`, and `getByTestId()` when they provide a stable and meaningful locator. Use XPath when it clearly helps with dynamic attributes, relationships, or complex DOM selection.